

Current High-Level Design

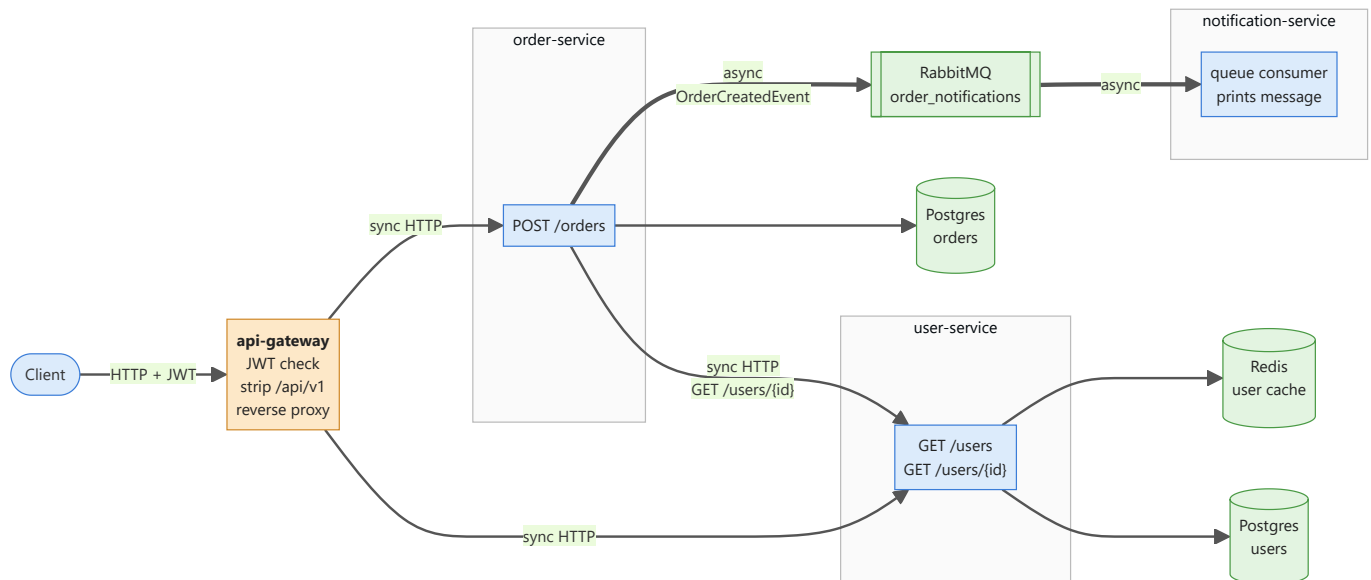
go-microservices · describes the code as it is on `main` (commit b962423) · 15 Sep 2026

1. What the system does today

A small order system split into four Go programs. A client can create an order for an existing user. The order is saved and the customer gets a (printed) confirmation message. User data is read-only: three users are pre-loaded when user-service starts, and there is no way to add, change or delete users.

Part	Job	Stores / uses	Talks to
api-gateway Gin, :8000	Single entry point. Checks the JWT, adds <code>X-User-ID</code> / <code>X-User-Role</code> headers, removes the <code>/api/v1</code> prefix, forwards the request.	Nothing	user, order, notification services (HTTP)
user-service net/http, :8001	Returns users: list all, or one by id.	Postgres <code>userdb</code> , Redis (10 min cache)	—
order-service net/http, :8002	Creates an order: checks the user exists, saves the order, sends an "order created" message.	Postgres <code>orderdb</code>	user-service (HTTP), RabbitMQ (publish)
notification-service no HTTP	Reads "order created" messages and prints a confirmation to its log. No email is actually sent.	Nothing	RabbitMQ (consume)

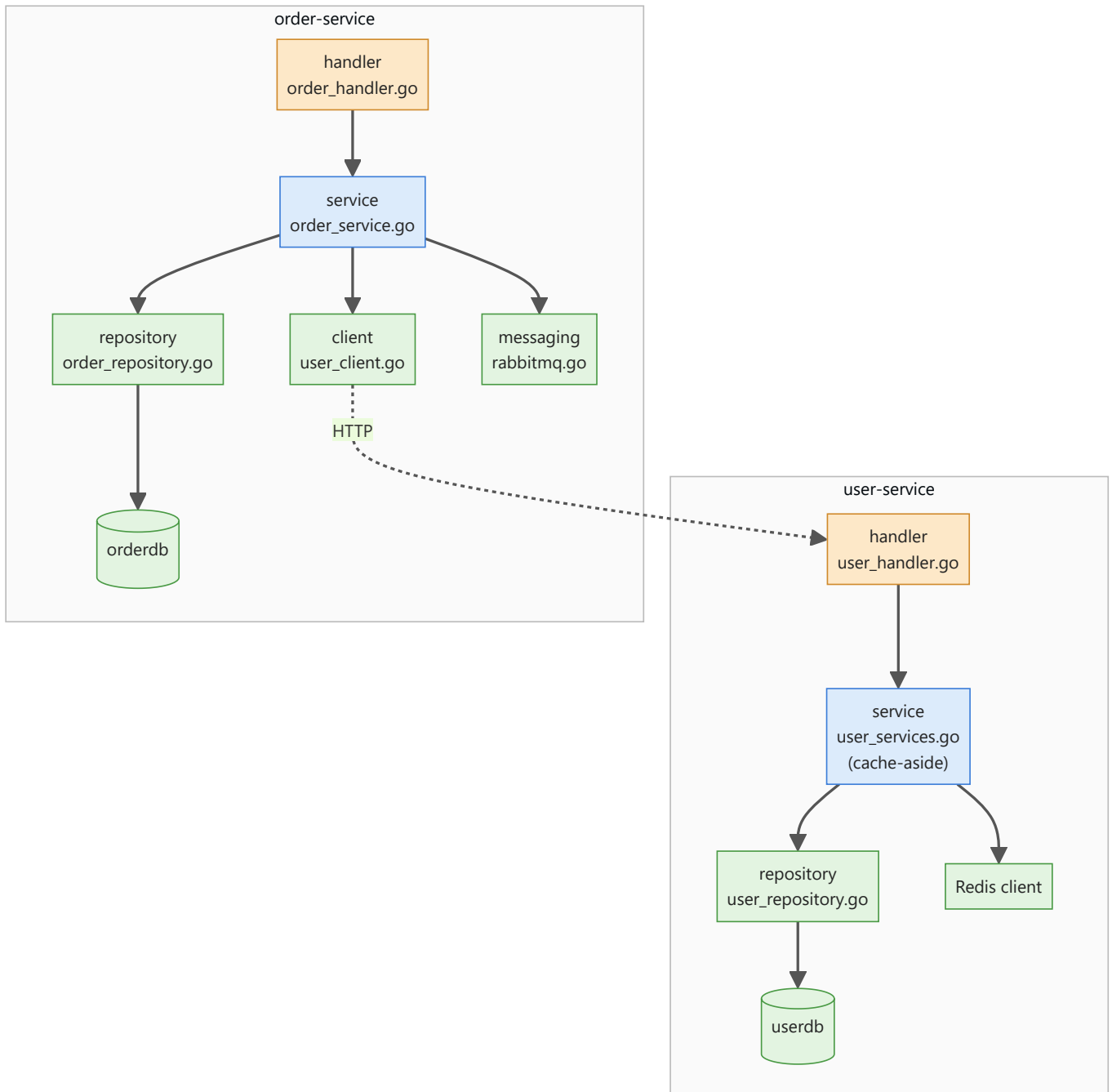
2. Component diagram



Thin arrows = request waits for a reply. Thick arrows = message sent through the queue, nobody waits.

3. Inside each service

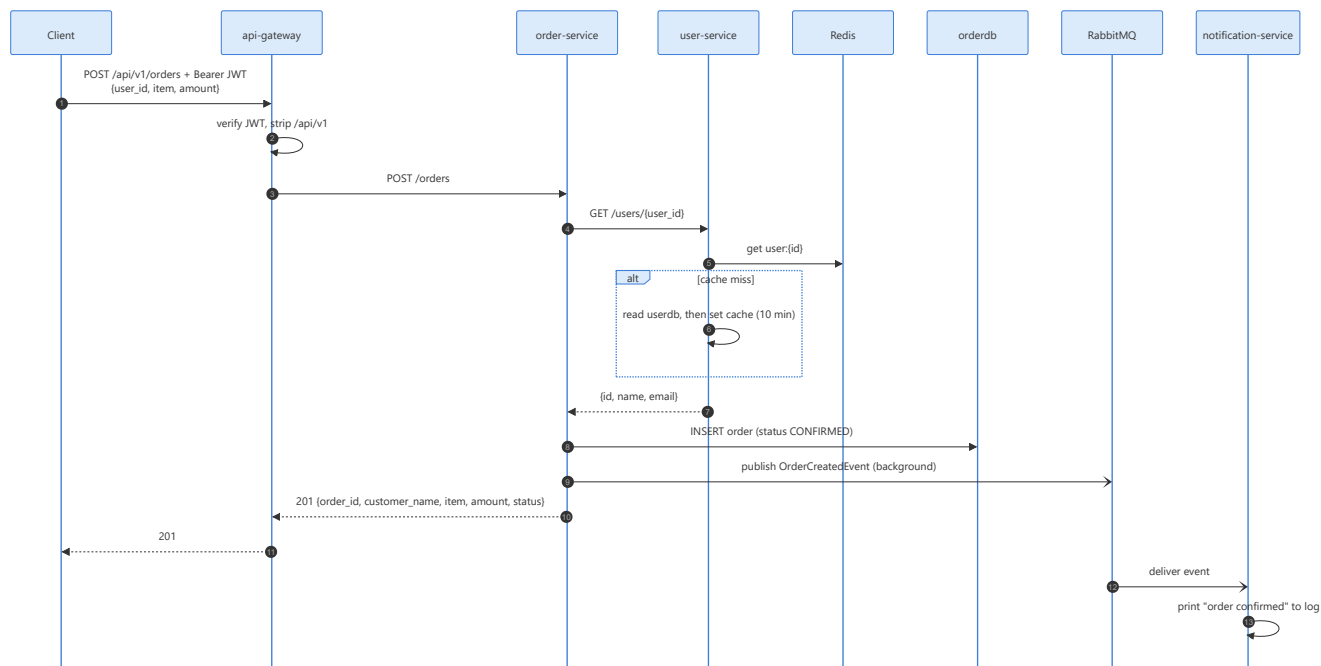
user-service and order-service follow the same layout: a **handler** reads the HTTP request, a **service** holds the logic, and a **repository** talks to the database. order-service adds a **client** for calling user-service and a **messaging** part for RabbitMQ. Everything is wired together by hand in `main.go`. order-service uses Go interfaces between layers (easy to swap in fakes for tests). user-service uses concrete types.



How a user is read (cache-aside)

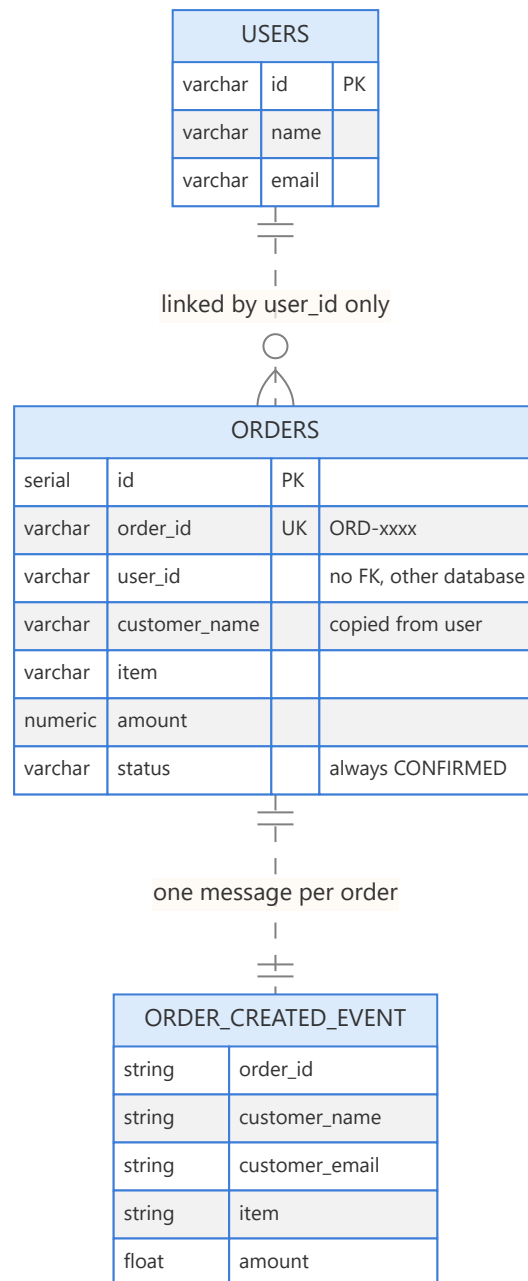
1. Look in Redis under `user:{id}` (or `users:all` for the list).
2. If found, return it.
3. If not, read from Postgres, save the result in Redis for 10 minutes, then return it.

4. Main flow: creating an order



If the user does not exist or user-service is down, the order is rejected with `400`. The client gets its reply right after the database insert. The message is sent in the background, so a failure there is only logged.

5. Data



- Each service owns its database. Nobody reads another service's tables. order-service copies the customer's name into the order at creation time.
- Tables are created in code at startup (`CREATE TABLE IF NOT EXISTS`). There are no migration files.
- The event struct is copied by hand into both order-service and notification-service. There is no shared definition, so changing one side can quietly break the other.

6. Routes: gateway vs. what services really handle

Gateway route	Auth	Forwarded as	Service handles it?
POST /api/v1/users/register	public	POST /users/register	No → 405
POST /api/v1/users/login	public	POST /users/login	No → 405
GET /api/v1/users/profile	JWT	GET /users/profile	Treated as user id "profile" → 404
PUT /api/v1/users/profile	JWT	PUT /users/profile	No → 405
ANY /api/v1/orders	JWT	/orders	POST only
ANY /api/v1/orders/*	JWT	/orders/*	No → 404
ANY /api/v1/notifications[/*]	JWT	notification-service:8003	No HTTP server → 502
<i>not exposed</i>	—	GET /users, GET /users/{id}	Exists, but only reachable inside Docker

Net result: **creating an order is the only feature that works through the gateway**, and only with a JWT made outside the system (nothing issues tokens).

7. Design gaps

Grouped by what they affect. These describe the current code, not criticism of the plan.

Things that break or give wrong results

- **No login.** The gateway requires a JWT, but no service creates one. Register/login routes lead nowhere.
- **Order ids clash.** `order_id = "ORD-" + (1000 + length of item name)`. Two orders whose item names have the same length get the same id. The `UNIQUE` column then rejects the second one.
- **Gateway identity is ignored.** The gateway passes `X-User-ID` from the token, but order-service takes `user_id` from the request body. Any logged-in user can place orders for any other user.
- **Lost notifications.** The message is sent in the background after the database insert, with no retry. The consumer uses auto-ack, so a message is marked done before it is handled. A crash at either point loses the notification with no trace.

Robustness

- order-service calls user-service with `http.Get`, which has no timeout. If user-service hangs, order requests hang too. The response body is not closed when the status isn't 200.
- Every service stops immediately (`log.Fatal`) if its database, Redis or RabbitMQ isn't ready at startup, and does not reconnect later. Docker's restart policy covers the first case, but a dropped RabbitMQ connection at runtime is not recovered.
- Only the gateway shuts down gracefully. The other services stop mid-request.
- The user cache is never cleared. That's fine today because users can't change, but it will return stale data once updates exist.

Configuration and code health

- Service addresses, database URLs and passwords are written directly in the Go code. Only the gateway reads them from environment variables.
- No automated tests. `npm test` just prints "No tests specified", so the CI pipeline always passes.
- notification-service exposes no `/metrics`, so it can't be monitored.
- Log messages mix English and Bengali.

Distance from the spec

`spec/project_details.md` describes a multi-tenant platform with tenant management, identity, a general transaction model, and analytics. The current code has none of the tenant or identity parts. It is a single-tenant proof of concept of the three-service pattern (gateway, services with their own data, messages through a queue). That pattern is a reasonable base for the spec, but most of the listed capabilities still need to be built.

8. Summary

Works today	Missing or broken
• One entry point with JWT checking • Each service has its own database • Redis cache for user reads • Orders trigger a message through RabbitMQ • Clean handler / service / repository layers • Metrics on gateway, user and order services	• Login / token issuing, user sign-up and editing • Most gateway routes don't match the services • Order id generation and trusting <code>user_id</code> from the body • Reliable message delivery • Timeouts, reconnects, config from environment • Tests, multi-tenancy